עבור כל שאלה יש להתייחס לדברים הבאים:

- ✓ במידה והיא מתקמפלת יש לכתוב מהו הפלט, כולל מה שיודפס לאחר הסוגריים המסולסלים שסוגרים את התוכנית
- ✓ במידה והשאלה אינה מתקמפלת, יש לכתוב מדוע
- ✓ במידה וישנה תעופה מהקוד, יש לכתוב מהו הפלט עד התעופה ולהסביר מדוע הקוד עף
- ✓ במידה ויש קוד שלא כל הזיכרון שוחרר, יש לציין זאת

שאלות 1 עד 4 יתייחסו לקטע הקוד הבא:

```
#include <iostream>
using namespace std;

class A
{
	static int idGenerator;
	int id;

public:
	A() : id(++idGenerator)
	{
		cout << "creating A id = " << id << endl;
	}

	A(const A& other) :id(other.id * 10)
	{
		cout << "Copying A id = " << id << endl;
	}

	virtual ~A()
	{
		cout << "destroying A id = " << id << endl;
		id--;
		idGenerator--;
	}

	operator int() const { return id; }
};

template <class T, class S>
class Tem1
{
	T t;
public:
	Tem1(const T& t=0) : t(t)
	{
		cout << "Creating Tem1\n";
	}

	operator const S() const { return (const S)t; }
};
```

```
template <class T, class S>
class Tem2 : public Tem1<T, S>
{
	S s;
public:
	friend ostream& operator<<(ostream& os, const Tem2& t)
	{
		os << t.s << end;
		return os;
	}

	const Tem2& operator=(const Tem1<T, S>& other)
	{
		if (this != &other)
		{
			Tem1<T, S>::operator=(other);
			s = other;
		}
		return *this;
	}

	const S& getS() const { return s; }
};

int A::idGenerator = 0;

void main1()
{
	Tem1<A, A> t;
	cout << t << endl;
}

void main2()
{
	A a;
	Tem1<A, A> t(a);
	cout << (A)t << endl;
}

void main3()
{
	Tem2<A, Tem1<A, int>> t1;
	Tem2<A, Tem1<A, int>> t2(t1);
	cout << t2 << endl;
}

void main4()
{
	Tem2<int, A> t2;
	Tem2<int, A> t1(t2);

	t2 = t2;
	cout << t2.getS() << endl;

}
```

שאלות 5 עד 9 יתייחסו לקטע הקוד הבא:

```
#pragma warning(disable: 4996)

#include <iostream>
using namespace std;

#include <string>
#include <vector>

class IceCream;

class IceCreamCombination
{
public:
	virtual void toOs(ostream& os) const = 0;

	friend ostream& operator<<(ostream& os, const IceCreamCombination& s)
	{
		s.toOs(os);
		return os;
	}

	virtual IceCreamCombination* clone() const = 0;
};

class IceCream : public IceCreamCombination
{
	char* flavour;
public:
	IceCream(const string& flavour)
	{
		this->flavour = new char[flavour.size() + 1];
		strcpy(this->flavour, flavour.c_str());
	}

	IceCream(const IceCream& other)
	{
		this->flavour = strdup(other.flavour);
		cout << "Copying IceCream\n";
	}

	IceCream(IceCream&& other) : flavour(nullptr)
	{
		swap(this->flavour, other.flavour);
		cout << "Moving IceCream\n";
	}

	~IceCream()
	{
		cout << "~IceCream " << (flavour ? flavour : "") << endl;
		delete[]flavour;
	}

	void changeFlavour(const char* newFlavour)
	{
		delete[]this->flavour;
		this->flavour = strdup(newFlavour);
	}
```

```
        virtual void toOs(ostream& os) const override {
                os << flavour << " ice cream";
        }

        operator const char*() { return flavour; }

        virtual IceCreamCombination* clone() const
        {
                return new IceCream(*this);
        }
};

class IceCreamTopping : public IceCreamCombination
{
        IceCreamCombination* theIceCream;
public:
        IceCreamTopping(IceCreamCombination* theIceCream)
                                : theIceCream(theIceCream) {}

        IceCreamTopping(const IceCreamTopping& other)
                                : IceCreamCombination(other)
        {
                theIceCream = other.theIceCream->clone();
        }

        virtual void toOs(ostream& os) const override
        {
                os << *theIceCream;
        }

        IceCreamCombination* getIceCreamCombination() { return theIceCream; }

        virtual IceCreamCombination* clone() const
        {
                return new IceCreamTopping(*this);
        }
};

class CreamTopping : public IceCreamTopping
{
        int grams;
public:
        CreamTopping(IceCreamCombination* theIceCream, int grams)
                            : IceCreamTopping(theIceCream), grams(grams) {}

        virtual void toOs(ostream& os) const override
        {
                IceCreamTopping::toOs(os);
                os << ", grams=" << grams;
        }

        void addGrams(int grams) { this->grams += grams; }
        int getGrams() const     { return grams; }

        virtual IceCreamCombination* clone() const
        {
                return new CreamTopping(*this);
        }
};
```

```
class ChocolateTopping : public IceCreamTopping
{
public:
	enum class eType {MILK, WHITE, DARK};
private:
	eType type;

	static const char * typeStr[3];
public:
	ChocolateTopping(IceCreamCombination* theIceCream, eType type)
				: IceCreamTopping(theIceCream), type(type) {}

	virtual void toOs(ostream& os) const override
	{
		IceCreamTopping::toOs(os);
		os << ", type=" << typeStr[(int)type];
	}

	virtual IceCreamCombination* clone() const
	{
		return new ChocolateTopping(*this);
	}
};

const char* ChocolateTopping::typeStr[3] = { "Milk", "White", "Dark" };

void main1()
{
	IceCreamCombination* s2 = new CreamTopping(
		new ChocolateTopping(
				new IceCream("vanilla"),
				ChocolateTopping::eType::MILK), 30);
	cout << *s2 << endl;

	cout << endl;
}

void main2()
{
	vector<IceCream> allIceCreams;
	allIceCreams.reserve(3);
	allIceCreams.push_back(IceCream("chocolate"));
	allIceCreams.push_back(IceCream("Stawberry"));
	CreamTopping ct(&allIceCreams[0], 40);
	cout << ct << endl;
}
```

```
void main3()
{
	vector<IceCreamCombination*> allIceCreams;
	allIceCreams.push_back(new IceCream("vanilla"));
	allIceCreams.push_back(new ChocolateTopping(
		new IceCream("chocolate"), ChocolateTopping::eType::WHITE));
	allIceCreams.push_back(new CreamTopping(new IceCream("chocolate"), 45));

	vector<IceCreamCombination*>::iterator itr = allIceCreams.begin();
	vector<IceCreamCombination*>::iterator itrEnd = allIceCreams.end();
	int sum = 0;
	for (; itr != itrEnd; ++itr)
	{
		CreamTopping* cb = dynamic_cast<CreamTopping*>(*itr);
		if (cb)
			sum += cb->getGrams();
	}

	cout << sum << endl;
}
```

```
void main4()
{
	IceCreamCombination* s1 = new CreamTopping(
		new ChocolateTopping(
			new IceCream("vanilla"),
				ChocolateTopping::eType::MILK), 30);

	cout << "1-----------\n";

	CreamTopping* sc1 =
		dynamic_cast<CreamTopping*>(dynamic_cast<IceCreamTopping*>
						(s1)->getIceCreamCombination());
	if (sc1 != nullptr)
	{
		sc1->addGrams(10);
		cout << *sc1 << endl;
	}
	cout << "2 ------------\n";

	ChocolateTopping* sc2 =
		dynamic_cast<ChocolateTopping*>(dynamic_cast<IceCreamTopping*>
						(s1)->getIceCreamCombination());
	if (sc2 != nullptr)
	{
		cout << *sc2 << endl;
	}
	cout << "3 ------------\n";

	CreamTopping* s3 = dynamic_cast<CreamTopping*>(s1->clone());
	IceCreamTopping* temp = dynamic_cast<IceCreamTopping*>
						(s3->getIceCreamCombination());
	IceCream* ic = dynamic_cast<IceCream*>(temp->getIceCreamCombination());
	if (ic)
	{
		ic->changeFlavour("banana");
		cout << *s1 << endl;
		cout << *s3 << endl;
		cout << *ic << endl;
	}

	delete s1;
	delete sc1;
	delete sc2;
	delete s3;
	cout << "4 ------------\n";
}
```